Algoritmo de Clustering DBSCAN

Aprendizaje No Supervisado

UDEM Jorge Alberto Ramírez García, M.Eng.

¿Qué es DBSCAN?

Density-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise (Ester et al., 1996)

Basado en densidad

- Un cluster es una región donde los puntos están apretados
- Separada de otras por zonas vacías

Formas arbitrarias

- Encuentra grupos curvos, alargados o anillos
- K-Means sólo encuentra grupos redondos

Detecta ruido

- Los puntos aislados quedan fuera (etiqueta -1)
- No obliga a asignar a todos

Sin k predefinido

- El número de clusters sale de los datos
- A cambio pide ε y min_samples

Densidad: ε-vecindad y alcanzabilidad

ε-vecindad

- Todos los puntos a distancia ≤ ε de p
- ε (eps) es un radio que tú eliges

Densamente alcanzable

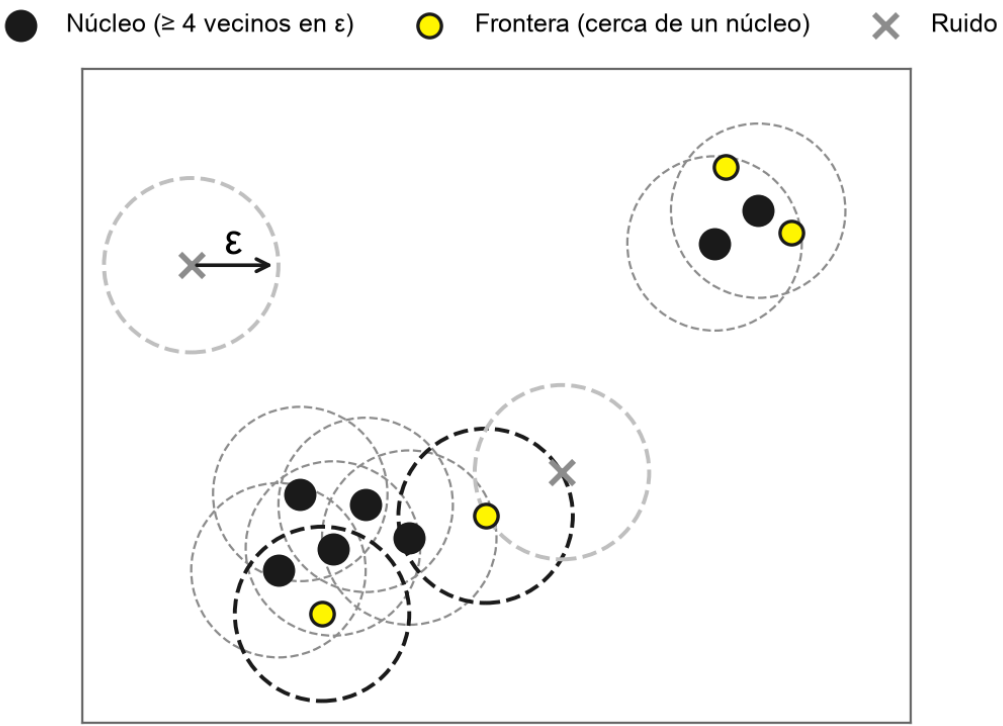
- q se alcanza desde p si hay una cadena de núcleos entre ellos
- Cada eslabón a menos de ε

Densamente conectados

- p y q se alcanzan desde un mismo núcleo
- Cluster = puntos conectados entre sí

$$N_\varepsilon(p) = \{ q : d(p, q) \le \varepsilon \} \qquad p \text{ es núcleo } \Leftrightarrow |N_\varepsilon(p)| \ge \text{min_samples}$$

Tres tipos de punto



Núcleo (core)

- ≥ min_samples puntos en su radio ε (él incluido)
- Siembra y hace crecer el cluster

Frontera (border)

- No es núcleo, pero está en el radio de uno
- Pertenece al cluster; no lo expande

Ruido (noise)

- Ni núcleo ni cerca de un núcleo
- Etiqueta -1: queda fuera

El algoritmo en 5 pasos

1. **Contar vecinos**
   Para cada punto, cuántos hay a distancia ≤ ε → marca los núcleos

2. **Abrir un cluster**
   Toma un núcleo sin cluster y crea uno nuevo

3. **Expandir**
   Agrega a sus vecinos; si un vecino es núcleo, agrega también a los suyos

4. **Repetir**
   Cuando ya no crece, vuelve al paso 2 con otro núcleo libre

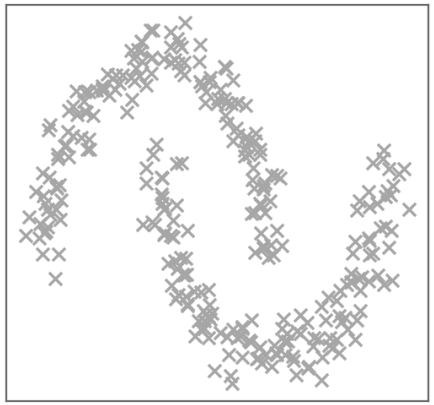
5. **Etiquetar el ruido**
   Lo que nunca se alcanzó desde un núcleo queda como -1

Diferencias con K-Means

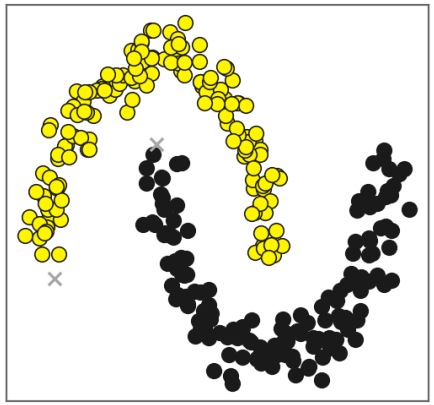
- No hay centroides ni iteraciones
- Núcleos y ruido son deterministas
- No hay predict: describe estos datos
- No necesita saber cuántos grupos hay

Los dos parámetros: ε y min_samples

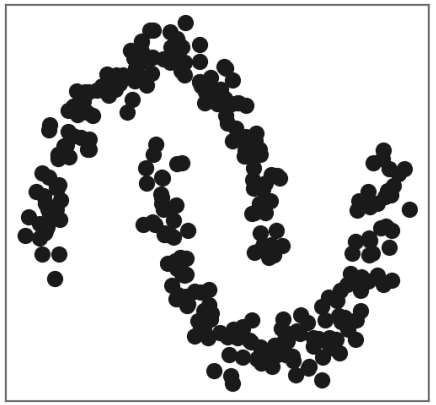
ε = 0.1 → ε muy chico: todo es ruido



ε = 0.3 → ε adecuado: 2 lunas



ε = 0.5 → ε muy grande: 1 solo cluster



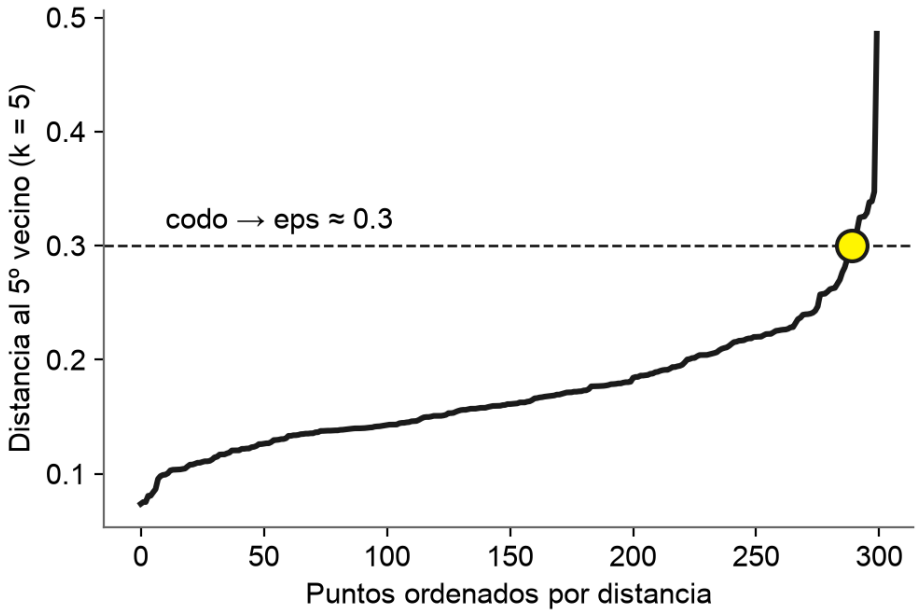
ε (eps): el radio

- Muy chico → todo es ruido o mil clusters diminutos
- Muy grande → un solo cluster (arriba: min_samples = 10)

min_samples (MinPts): la densidad

- Regla práctica: ≥ dimensiones + 1; arranque 2 × dim
- Más alto → más exigente → más ruido

Cómo elegir ε: la gráfica de k-distancia

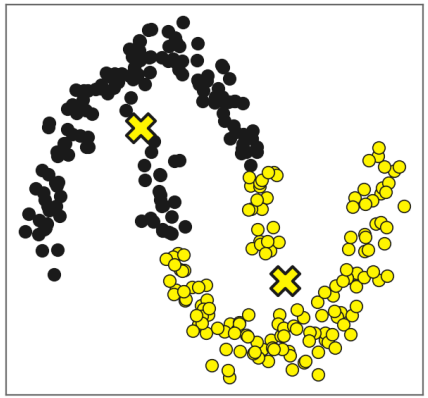


1. **Fija min_samples primero**
   ≥ dimensiones + 1 (2 × dim para arrancar)
2. **Distancia al k-ésimo vecino**
   Para cada punto, con k = min_samples
3. **Ordena y grafica**
   Dentro de un cluster: distancias chicas
4. **Busca el codo**
   Donde la curva se dispara: candidato a ε

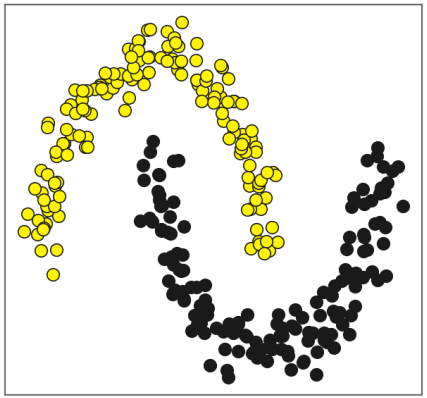
El codo es un punto de partida: afina con una grilla de ε × min_samples y mira siempre el % de ruido.

DBSCAN vs K-Means

K-Means (k = 2): corta con una recta



DBSCAN: sigue la densidad

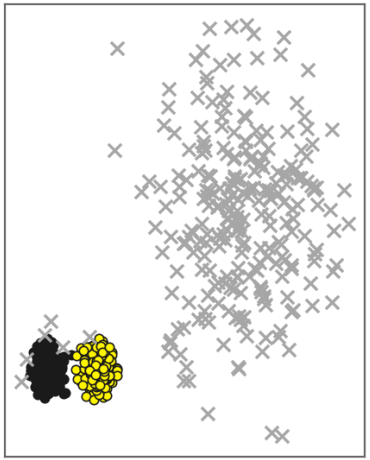


	K-Means	DBSCAN
¿Pide k?	Sí	No
Forma	Redonda	Cualquiera
Outliers	Los asigna	Ruido (-1)
Parámetros	k	ε, min_samples
Punto débil	Formas raras	Densidades distintas
Puntos nuevos	predict	Volver a ajustar

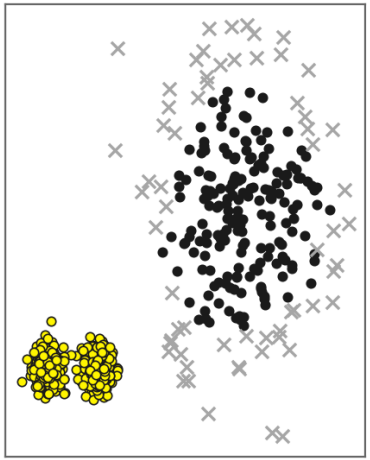
En las lunas, DBSCAN acierta (ARI = 1.0) y K-Means no (ARI = 0.48)… pero la silhouette favorece a K-Means (0.49 vs 0.38): premia grupos redondos. Mira siempre la gráfica.

Densidades variables: HDBSCAN

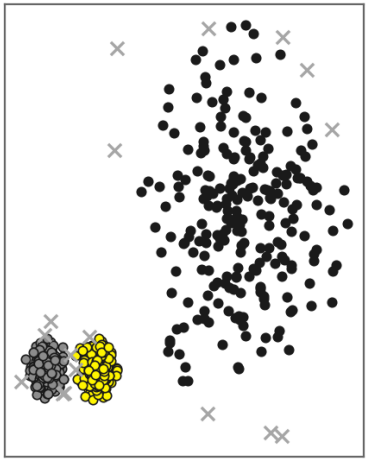
DBSCAN ε = 0.1: el disperso es ruido



DBSCAN ε = 0.3: se juntan los densos



HDBSCAN: los 3 grupos



El problema

- Un solo ε para todo el mapa
- ε chico pierde lo disperso; ε grande junta lo denso

HDBSCAN (sklearn.cluster.HDBSCAN)

- Prueba todos los radios y se queda con los clusters más estables
- Parámetro: min_cluster_size (tamaño mínimo de grupo)

Distancias: Euclidiana, Manhattan y Haversine

Euclidiana

- $d = \sqrt{\Sigma (p_i - q_i)^2}$
- Línea recta; el default
- Variables escaladas

Manhattan (L1)

- $d = \Sigma |p_i - q_i|$
- Suma de diferencias
- Útil en muchas dimensiones

Haversine

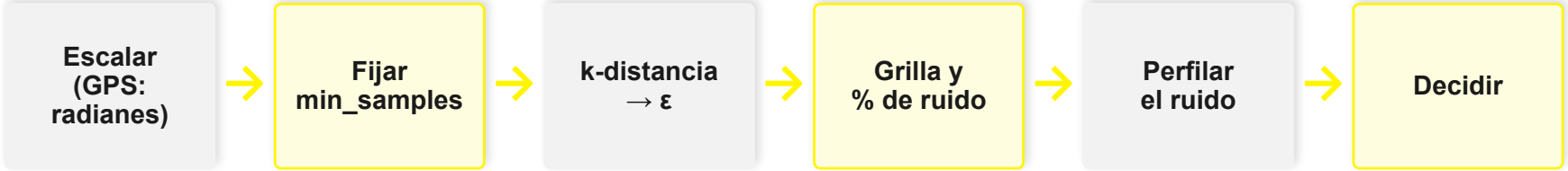
- Distancia sobre una esfera
- Coordenadas GPS
- metric='haversine'

Ejemplo en Monterrey (25.7° N)

- 1° de latitud ≈ 111 km, pero 1° de longitud ≈ 100.3 km: la euclidiana sobre grados distorsiona (y más lejos del ecuador)
- Macroplaza → Valle Oriente ≈ 4.9 km con Haversine

```
X = np.radians(df[['lat', 'lon']])     eps = km / 6371
```

Flujo de trabajo



Errores comunes

- Silhouette con el ruido incluido
- Elegir el modelo sólo por silhouette: más ruido la infla
- ε sin escalar las variables
- Haversine con grados en lugar de radianes
- Contar el -1 como un cluster

Buenas prácticas

- Reporta silhouette sin ruido junto con el % de ruido
- Compara modelos sobre el mismo conjunto de puntos
- El ruido no es basura: perfílalo antes de decidir
- Prueba la estabilidad: ¿cambia todo con un ε cercano?

Código mínimo

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler


X = StandardScaler().fit_transform(df[cols])


plot_k_distancia(X, k=6)    # elige eps


dbscan = DBSCAN(
    eps=0.5,
    min_samples=6
).fit(X)


labels = dbscan.labels_       # -1 = ruido
evaluar_clustering('DBSCAN', X, labels)
```

```
# Coordenadas GPS
import numpy as np


X_rad = np.radians(df[['lat', 'lon']])


dbscan_gps = DBSCAN(
    eps=0.5 / 6371,       # 0.5 km
    min_samples=10,
    metric='haversine'
).fit(X_rad)


# Sin eps: HDBSCAN
from sklearn.cluster import HDBSCAN
HDBSCAN(min_cluster_size=20).fit(X)
```

Casos de uso

Geoanalítica

- Zonas de alta demanda para centros de distribución
- Paradas de transporte, zonas de criminalidad

Ciberseguridad

- Tráfico de red anómalo
- Patrones de ataque como ruido

Clientes

- Segmentos densos + clientes que no encajan
- Atención personalizada a los atípicos

Bioinformática

- Secuencias y células con formas complejas
- Grupos sin forma redonda

Conclusiones

1. DBSCAN agrupa por densidad: núcleos, frontera y ruido, sin pedir k
2. Fija min_samples primero y elige ε con la gráfica de k-distancia
3. Reporta la silhouette sin ruido junto con el % de ruido
4. Densidades muy distintas → HDBSCAN; coordenadas GPS → Haversine en radianes
5. Escala antes de medir distancias, y perfila el ruido antes de descartarlo